



AFX Cluster reinit script - public version

{toc:printable=true|style=disc|maxLevel=3|minLevel=1|type=list}

Latest version: `reinit_afx_v2.py`

Updated 5/20/2026

Last version: `reinit_afx_v1.py`

Table of contents

✓ Click here to expand...

- [Overview](#)
- [What's New in v2](#)
- [Prerequisites](#)
- [Supported Operating Systems](#)
- [Required Packages and Modules](#)
- [Network Requirements](#)
- [Configuration File](#)
- [Operation Modes](#)
- [LOADER Commands Reference](#)
- [Command-Line Reference](#)
- [Step-by-Step Instructions](#)
- [Session Logging](#)
- [Debug Mode](#)
- [Screen Mode](#)
- [Background Mode](#)
- [Known Issues and Gotchas](#)
- [Troubleshooting](#)
- [Changelog](#)

Overview

Starting with ONTAP 9.19.1, Storage Availability Zones (SAZs) support 96-bit filesystem semantics, enabling storage capacity larger than 16 PB per single zone. To gain the benefits of the new 96-bit SAZ, the existing 64-bit SAZ must be destroyed (along with any data it contains) and the entire cluster reinitialized. There is no way to simply convert the existing SAZ in-place.

This process involves many sequential and parallel steps — including wait times between operations — that benefit greatly from automation to reduce human error and minimize hands-on time.

reinit_afx_v2.py is an automated console management script that assists NetApp field engineers and storage administrators with reinitializing NetApp AFX cluster nodes via the BMC (Baseboard Management Controller) / Service Processor (SP) console. It is the second-generation rewrite of the original `afx_reinit_v1.py`.

The script automates the following core tasks:

- Connects to the BMC/SP via SSH
- Validates BMC/SP status and existing session conflicts
- Performs a system reset or power cycle as needed
- Enters the system console and interrupts the AUTOBOOT sequence
- Executes LOADER-level boot configuration commands
- Selects the appropriate boot menu option
- Drives the ONTAP cluster setup wizard in fully automated mode
- Adds peer nodes to an existing cluster (sequentially or in parallel)
- Manages ONTAP software upgrades via rolling takeover/giveback
- Installs ONTAP via netboot
- Configures passwordless SSH access to cluster management
- Creates and saves cluster configuration backups
- Verifies BMC authentication

All session activity is captured in a timestamped log directory with a human-readable summary report.

What's New in v2

Feature	Description
JSON Config File	Cluster and node credentials can be pre-supplied in a JSON config file, eliminating repeated prompts across multi-node operations.
Full Automation Modes	Modes 1b, 2b, and 3 drive the ONTAP cluster setup and node-join wizards without operator interaction.
Parallel Node Operations	Mode 2b and Mode 3 run peer node additions in parallel threads, significantly reducing multi-

	node reinit time.
End-to-End Mode (3)	Combines 1b (primary init) + 2b (peer adds) into a single unattended run.
ONTAP Upgrade (4a)	Rolling upgrade via automated takeover/giveback sequence.
Netboot Install (4b)	Automated ONTAP netboot and software installation.
SSH Key Setup (4d)	Configures passwordless SSH from the script host to cluster management.
Config Backup (4e)	Saves or constructs cluster configuration (cluster name, IPs, licenses, nodes) to a JSON file for use in future runs.
BMC Auth Verify (4f)	Batch-tests BMC SSH credentials for all nodes in the config file.
Session Logging	Captures per-phase and per-step timing, outcome (PASS/FAIL/WARN), and a complete warning and error inventory in the summary file.
Background Mode	--bg flag: handles SIGHUP cleanly so the script can run unattended in a detached or screen session.
Screen Mode	--screen flag: automatically re-launches the script inside a detached GNU screen session. Protects against SSH disconnections and terminal timeouts. Implies --bg.
Node add resume	Resumes interrupted node add processes
Physical disk zeroing	Adds option to physically zero disks rather than fast zero (which helps ensure performance consistency)

Prerequisites

Before running this script, ensure the following are in place:

- Python 3.6 or later installed on the client machine
- SSH access to all BMC/Service Processor addresses
- BMC/SP credentials are known (username and password)
- BMC/SP addresses are reachable from the client (port 22/TCP)
- Cluster management IP and credentials are known (for modes that interact with ONTAP)
- For config-file-driven runs: a valid `reinit-config.json` is prepared (see Configuration File section)

BMC/Service Processor Configuration

The BMC/Service Processor must be configured and accessible over the network before running this script. Refer to the official NetApp documentation:

- <https://docs.netapp.com/us-en/ontap/system-admin/manage-sp-bmc-concept.html>
- <https://docs.netapp.com/us-en/ontap/system-admin/configure-sp-bmc-network-concept.html>

Supported Operating Systems

The script has been tested on CentOS 7.x, Red Hat 9.x, and Ubuntu 22.04. It should work on any system that supports Python 3.6+.

OS	Tested Versions	Package Manager
Red Hat Enterprise Linux (RHEL)	7.x, 8.x, 9.x	yum / dnf
CentOS	7.x, 8.x	yum / dnf
Fedora	Current	dnf
Ubuntu	18.04, 20.04, 22.04, 24.04	apt
Debian	10, 11	apt
macOS	Catalina and later	pip only
Windows	10, 11 (with Python installed)	pip only

The script automatically detects the operating system and uses the appropriate package manager (`apt` , `dnf` , or `yum`) for installing system-level dependencies. On macOS and Windows, `pip` is used exclusively.

Required Packages and Modules

Python Modules

Module	Purpose	Install Method
paramiko	SSH connectivity to BMC/SP and cluster	Auto-installed by script if not present

If `paramiko` is missing the script detects it at startup and prompts you to install it:

- Ubuntu/Debian: `sudo apt install python3-paramiko`
- RHEL/CentOS/Fedora: `sudo dnf install python3-paramiko` or `sudo yum install python3-paramiko`
- Fallback (all OS): `pip install paramiko`

Standard Library Modules (no install required)

`subprocess` , `sys` , `os` , `time` , `re` , `getpass` , `logging` , `threading` , `signal` , `argparse` , `platform` , `socket` , `warnings` , `datetime` , `json` , `atexit`

Network Requirements

Port Requirements

Port	Protocol	Direction	Purpose
22	TCP	Client → BMC/SP	SSH connection to each node's BMC or Service Processor
22	TCP	Client → Cluster Mgmt IP	SSH connection to ONTAP cluster management (modes 4a–4f)

Firewall Configuration

Ensure that port 22 (SSH) is open outbound from the client machine to all BMC/SP addresses and to the cluster management IP.

Linux (firewalld)

```
1 # Check firewalld status
2 sudo systemctl status firewalld
3
4 # Temporarily disable (re-enables on reboot)
5 sudo systemctl stop firewalld
6
7 # Re-enable after the procedure
8 sudo systemctl start firewalld
```

Linux (iptables)

```
1 # Check current rules
2 sudo iptables -L OUTPUT -n
3
4 # Allow outbound SSH if blocked
5 sudo iptables -A OUTPUT -p tcp --dport 22 -j ACCEPT
```

SELinux

SELinux typically does not block outbound SSH. If issues occur:

```
1 # Check status
2 getenforce
3
4 # Temporarily set Permissive (reverts on reboot)
5 sudo setenforce 0
6
7 # Re-enable after procedure
8 sudo setenforce 1
```

 Do not permanently disable SELinux on production systems.

Connectivity Test

Before running the script, verify that you can reach each BMC:

```
1 # Test SSH connectivity
2 ssh admin@<bmc-address>
3
4 # Test port connectivity
5 nc -zv <bmc-address> 22
```

Configuration File

The script accepts a JSON configuration file that pre-fills cluster and node parameters. This eliminates repeated prompts during multi-node runs and enables fully unattended automation.

Auto-Discovery

The script automatically searches for config files in the following locations (in order):

1. `configs/reinit-config.json` (subdirectory next to the script)
2. `reinit-config.json` (same directory as the script)
3. Current working directory

The following filenames are recognized: `reinit-config.json`, `reinit_config.json`, `reinit-afx-config.json`, `reinit_afx_config.json`, `afx-reinit-config.json`, `config.json`

You can also specify the path explicitly:

```
1 | python3 reinit_afx_v2.py --config /path/to/myconfig.json
```

Config File Schema

```
1 | {
2 |   "cluster": {
3 |     "name": "cluster-name",
4 |     "clus_mgmt_address": "192.168.1.100",
5 |     "clus_mgmt_mask": "255.255.255.0",
6 |     "clus_mgmt_gw": "192.168.1.1",
7 |     "clus_mgmt_port": "e0M",
8 |     "user": "admin",
9 |     "password": "password",
10 |     "dns_domains": "example.com",
11 |     "dns_servers": "192.168.1.10,192.168.1.11",
12 |     "location": "Rack 1",
13 |     "contact": "admin@example.com"
14 |   },
15 |   "primary_node": {
16 |     "bmc": "192.168.2.10",
17 |     "bmc_user": "admin",
18 |     "bmc_password": "NetApp1!AFX",
19 |     "node_mgmt_port": "e0M",
20 |     "node_mgmt_ip": "192.168.2.11",
21 |     "node_mgmt_netmask": "255.255.255.0",
22 |     "node_mgmt_gateway": "192.168.2.1"
23 |   },
24 |   "secondary_nodes": [
25 |     {
26 |       "bmc": "192.168.2.20",
27 |       "bmc_user": "admin",
28 |       "bmc_password": "NetApp1!AFX",
29 |       "node_mgmt_port": "e0M",
30 |       "node_mgmt_ip": "192.168.2.21",
31 |       "node_mgmt_netmask": "255.255.255.0",
32 |       "node_mgmt_gateway": "192.168.2.1"
33 |     }
34 |   ]
35 | }
```

Field Behavior

Field value in JSON	Runtime behavior
---------------------	------------------

Field omitted (key not present)	Script prompts the operator at runtime
Field set to "" (empty string)	Used as-is with no prompt. For passwords this means "no password".
Field set to a non-empty value	Used directly, no prompt

Print a ready-to-edit example config at any time:

```
1 | python3 reinit_afx_v2.py --config-example
```

Config File Schema — Mode 4e Output

Mode 4e writes a single JSON object with up to four top-level keys.

Field	Source	Notes
name	Gathered from cluster or prompted	Cluster name
clus_mgmt_address	Gathered / prompted	Cluster management IP
clus_mgmt_mask	Gathered / prompted	Cluster management netmask
clus_mgmt_gw	Gathered / prompted	Cluster management gateway
clus_mgmt_port	Prompted (build) / mgmt_port (gather)	e.g. e0M
user	Not set by 4e — preserved from existing file only	Cluster admin username
dns_domains	Gathered / prompted	Comma-separated domain list
dns_servers	Gathered / prompted	Comma-separated server IPs
location	Gathered / prompted	Optional
contact	Gather path only	Optional

The `secondary_nodes` array elements are populated from the "add extra nodes" prompt in the gather or build/create path.

Operation Modes

The script presents a menu at startup:

```
1 NetApp AFX BMC Console Automation
2
3 [1a] Initialize First Node (interactive)
4 [1b] Initialize First Node (automated)
5 [2a] Add Node to Cluster (interactive)
6 [2b] Add Node to Cluster (automated)
7 [2c] Resume interrupted node additions.
8 [3] End-to-End Reinit (1b + all 2b nodes)
9 [4a] ONTAP Upgrade
10 [4b] Netboot Install
11 [4c] License Install
12 [4d] SSH Key Setup
13 [4e] Config Backup
14 [4f] BMC Auth Verify
15
16 Select mode:
```

Mode	Short Name	Description
1a	Initialize First Node (interactive)	Boots to LOADER, sets <code>destroy-all-storage-pods</code> flag, selects boot menu option 9 (Clean System Configuration). Prompts the operator for all cluster setup wizard inputs.
1b	Initialize First Node (automated)	Same as 1a, but drives the full ONTAP cluster setup wizard automatically using values from config file or prompts.
2a	Add Node to Cluster (interactive)	Boots to LOADER, selects boot menu option 4 (Initialize and configure system). Operator completes the node-join wizard.
2b	Add Node to Cluster (automated)	Same as 2a, but drives the node-join wizard automatically. Supports adding multiple secondary nodes in parallel.

2c	Resume Node Additions	Resumes interrupted node-join operations from the last successful checkpoint. Use when a previous mode 2b or mode 3 run was interrupted before all secondary nodes completed.
3	End-to-End Auto Reinit	Runs mode 1b on the primary node, then runs mode 2b on all secondary nodes in parallel. Fully unattended with a config file.
4a	ONTAP Upgrade	Performs a rolling upgrade of both nodes via automated takeover, software update, and giveback sequence.
4b	Netboot Install	Boots a node via the network and installs a new ONTAP image from a netboot server.
4c	License Install	Installs ONTAP licenses on an existing cluster.
4d	SSH Key Setup	Configures passwordless SSH from the script host to the cluster management interface.
4e	Config Backup	Connects to the cluster and captures its current configuration (name, IPs, licenses, nodes) to a JSON file. Can also build a config file manually from user prompts.

4f	BMC Auth Verify	Tests BMC SSH authentication for all nodes defined in the config file and reports pass/fail.
----	-----------------	--

LOADER Commands Reference

Mode	LOADER Commands
1a / 1b	<code>set-defaults</code> , <code>setenv bootarg.destroy.all.storage.pods true</code> , <code>saveenv</code> , <code>boot_ontap menu</code> → Option 9
2a / 2b/2c	<code>set-defaults</code> , <code>saveenv</code> , <code>boot_ontap menu</code> → Option 4
4b	<code>set-defaults</code> , <code>setenv AUTOBOOT false</code> , <code>saveenv</code> , netboot sequence

Command-Line Reference

```
1 | python3 reinit_afx_v2.py [OPTIONS]
```

Option	Short	Description
<code>--config PATH</code>	<code>-c PATH</code>	Path to a JSON config file. If omitted, the script auto-discovers config files or prompts for all values.
<code>--config-example</code>		Print an annotated example config file and exit.
<code>--debug</code>	<code>-d</code>	Enable debug mode: print all raw console I/O to the screen. Also enables verbose Paramiko SSH logging.

<code>--bg</code>		Background mode: handle SIGHUP so the log is closed cleanly when the terminal closes. Use with <code>nohup</code> or <code>screen</code>
<code>--screen</code>		Re-launch the script inside a detached GNU screen session. Keeps the run alive if your SSH connection drops or times out. Implies <code>--bg</code> . Use <code>screen -r afx-reinit</code> to reattach. No-op if already running inside screen.
<code>--help, --h</code>		Show a short man page about the script's options.

Step-by-Step Instructions

Step 1: Download and Place the Script

Place `reinit_afx_v2.py` on the client machine that has network access to all BMC/SP addresses and the cluster management IP.

```

1 # Recommended: create a dedicated directory
2 mkdir ~/afx-reinit
3 cp reinit_afx_v2.py ~/afx-reinit/
4 cd ~/afx-reinit

```

Step 2: (Optional) Create a Config File

For automated or multi-node runs, create a `reinit-config.json` in the same directory:

```

1 # Print the example format
2 python3 reinit_afx_v2.py --config-example > configs/reinit-config.json
3 # Edit with your cluster and node details
4 vi configs/reinit-config.json

```

Step 3: Run the Script

```

1 # Standard interactive run
2 python3 reinit_afx_v2.py
3
4 # With explicit config file
5 python3 reinit_afx_v2.py --config configs/reinit-config.json
6

```

```
7 # With debug output
8 python3 reinit_afx_v2.py --debug
9
10 # Auto-launch in screen (recommended for remote/SSH sessions)
11 python3 reinit_afx_v2.py --screen --config configs/reinit-config.json
12 # Reattach later with: screen -r afx-reinit
13
14 # In background via nohup (alternative to --screen)
15 nohup python3 reinit_afx_v2.py --bg --config configs/reinit-config.json > nohup.out 2>&1 &
```

What happens at startup:

- The script checks for required Python modules (`paramiko`); installs if missing.
- If a config file is found, you are prompted to use it or enter values manually.
- A session log directory is created under `logs/YYYYMMDD_HHMMSS/` .

Step 4: Select an Operation Mode

The script presents a numbered menu. Enter the number corresponding to the desired mode. See Operation Modes for a full description of each option.

Step 5: Enter Credentials (if not in config file)

If no config file was loaded (or if fields were left blank), the script prompts for:

- BMC/SP hostname or IP address
- BMC/SP username and password
- Cluster management IP, username, and password (where applicable)
- Node management port, IP, netmask, gateway

Step 6: BMC Connection and Validation

The script establishes an SSH connection to the BMC/SP and waits for the initial prompt. If an existing session is detected:

- *Interactive mode: you are asked whether to disconnect the existing session.*
- *Automated mode (modes 1b, 2b, 3): the existing session is automatically disconnected.*

Step 7: System Reset

The script issues a system reset (or power cycle) command to the controller. It then waits for the console to become active.

The script watches for expected output patterns at each stage. If a pattern is not seen within the timeout window, an error is logged and the script either retries or prompts the operator.

Step 8: AUTOBOOT Interruption

Once the LOADER prompt appears, the script:

- Sends the appropriate `set-defaults` and `setenv` commands
- Calls `saveenv` to persist the settings
- Issues `boot_ontap menu` to display the boot menu
- Selects the appropriate boot menu option (option 4 or option 9)

Step 9: Post-Boot Wizard

Depending on the mode:

- *1a (interactive): The script provides a live terminal passthrough. The operator answers wizard questions manually.*
- *1b / 2b / 3 (automated): The script drives the wizard using config file values or pre-supplied prompts. No operator interaction is required once the run starts.*

Step 10: Multi-Node Parallel Operations (modes 2b and 3)

In mode 2b and mode 3, secondary nodes are processed in parallel worker threads. The script monitors each thread and aggregates results. Each node reports independently to the session log.

Step 11: Exit and Review Logs

When the script completes, it prints the path to the log directory and a brief summary. Review the summary file for timing, warnings, and errors.

Session Logging

All output is captured in a timestamped log directory:

```
1 logs/  
2   YYYYMMDD_HHMMSS/  
3     session_<label>.log    <- full raw console transcript  
4     summary_<label>.log    <- human-readable summary report
```

The `logs/` directory is created in the same folder as the script.

Summary File Format

The summary file contains:

- **Result: PASS, PASS (with warnings), or FAIL**
- **Phase Timing: duration of each named phase (e.g., "BMC Connect", "LOADER", "Wizard")**

- **Step Timing:** duration of individual steps within each phase
- **Warnings (N):** timestamp and message for each warning logged during the run
- **Errors (N):** timestamp and message for each error logged during the run

Example summary:

```

1  =====
2  SESSION SUMMARY - Mode 1b: Initialize First Node (automated)
3  Result : PASS (1 warning)
4  =====
5
6  Phase Timing
7    BMC Connect      :   3.2s
8    System Reset     :  12.4s
9    LOADER            :  18.1s
10   Wizard            : 142.7s
11   Total              : 176.4s
12
13  Step Timing
14    wait_bmc_prompt   :   3.2s
15    send_reset        :   0.1s
16    wait_autoboot     :  12.3s
17    ...
18
19  Warnings (1)
20    2026-04-07 14:23:01 Existing BMC session detected - auto-disconnected
21
22  Errors (0)
23    (none)
24  =====

```

Debug Mode

Enable with `--debug` or `-d`.

In debug mode:

- All raw console I/O (BMC and ONTAP) is printed directly to the terminal in addition to being written to the log file.
- Python logging is set to DEBUG level, showing verbose Paramiko SSH negotiation and channel activity.
- Useful for diagnosing unexpected hangs, mismatched prompt patterns, or SSH authentication issues.

```
1 python3 reinit_afx_v2.py --debug
```

Screen Mode

Enable with `--screen`.

When `--screen` is specified the script checks whether it is already running inside a GNU screen session (via the `STY` environment variable). If not, it:

1. Verifies that `screen` is installed (exits with install instructions if missing)
2. Strips `--screen` from the argument list to prevent recursion
3. Appends `--bg` so the log is flushed cleanly on detach
4. Spawns: `screen -dmS afx-reinit python3 reinit_afx_v2.py --bg [other args]`
5. Prints the reattach command and exits the outer process

The script then runs entirely inside the screen session. If your SSH connection drops, the run continues uninterrupted. Reconnect to the host and reattach:

```
1 # Launch in screen
2 python3 reinit_afx_v2.py --screen --config configs/reinit-config.json
3
4 # Reattach after reconnecting
5 screen -r afx-reinit
6
7 # List active sessions
8 screen -ls
```

 `--screen` implies `--bg`. You do not need to specify both flags.

`--screen` is a no-op if you are already inside a screen (or tmux) session — the script detects this and continues normally without spawning a child session.

GNU screen must be installed on the client machine. If it is missing, the script will print install instructions and exit cleanly.

Background Mode

Enable with `--bg`.

Registers a SIGHUP handler so the session log is flushed and closed cleanly when the controlling terminal disconnects. Use this when running the script via `nohup`, `screen`, or `tmux`

 For the most convenient experience with screen, use `--screen` instead — it handles session creation automatically. See Screen Mode.

```
1 # Using nohup
2 nohup python3 reinit_afx_v2.py --bg --config configs/reinit-config.json > nohup.out 2>&1 &
3
4 # Manually launching inside screen
5 screen -S afx-reinit python3 reinit_afx_v2.py --bg --config configs/reinit-config.json
6 # Detach with Ctrl+A, D
7 # Reattach with: screen -r afx-reinit
```

Known Issues and Gotchas

- *BMC session timeout: Some BMC firmware versions disconnect idle sessions after 5–10 minutes. If the script appears to hang waiting for the LOADER prompt after a long delay, try re-running with a fresh BMC session.*
- *Boot menu timing: The window for interrupting AUTOBOOT is narrow. The script attempts the interrupt character as soon as it detects the AUTOBOOT countdown. If the system boots fully before the interrupt is sent, the script will report an error. Reset the node and re-run.*
- *ONTAP wizard timeouts: The ONTAP cluster setup wizard occasionally pauses for DNS lookups or license validation. The script uses generous timeouts for these steps but may time out on very slow networks. Run with `--debug` to observe wizard progress in real time.*
- *Parallel node adds: In modes 2b and 3, all secondary nodes are started simultaneously. If one node fails, the others continue running. Check the summary log for per-node results.*
- *Config file and empty string fields: Setting a password field to "" in the config file means the script will send an empty password (no prompt). This is intentional for BMCs that use passthrough credentials. Do not set "" for fields that require real values.*
- *Windows: The `--bg` SIGHUP handler is a no-op on Windows (SIGHUP is not supported). The script still runs correctly; the warning can be ignored. The `--screen` flag is also unavailable on Windows (GNU screen is Linux/macOS only); use WSL or a Linux jump host for long-running sessions.*

Troubleshooting

Script fails immediately with "ModuleNotFoundError: No module named 'paramiko'"

The script should auto-detect this and prompt to install. If it does not:

```
1 pip install paramiko
2 # or
3 sudo apt install python3-paramiko # Ubuntu/Debian
4 sudo dnf install python3-paramiko # RHEL/Fedora
```

"Connection refused" or "SSH timeout" connecting to BMC

- Verify the BMC address is correct and reachable: `ping <bmc-address>`
- Verify port 22 is open: `nc -zv <bmc-address> 22`
- Verify firewall rules on the client (see Network Requirements)
- Verify the BMC is configured and powered on (access IPMI or iLO/iDRAC console if possible)

"Authentication failed" when connecting to BMC

- Double-check the BMC username and password

- Some BMC firmware defaults to admin / admin; others use ADMIN / ADMIN
- The script supports empty passwords (for BMCs with no password configured) by setting `bmc_password: ""` in the config file
- Run mode 4f (BMC Auth Verify) to test credentials for all nodes without starting a reinit

Script hangs waiting for LOADER prompt

- Enable `--debug` to see raw console output
- The system may be taking longer than expected to POST
- Some systems require the boot interrupt character multiple times — the script retries automatically
- If the system has already booted past LOADER, perform a manual reset and re-run

ONTAP cluster wizard not progressing

- Run with `--debug` to watch the wizard in real time
- Check that cluster management IP and gateway values are reachable from the cluster node's management port
- Verify DNS server addresses in the config file are reachable

"UnboundLocalError" or Python traceback

- Ensure you are using Python 3.6 or later: `python3 --version`
- Confirm the script file was not corrupted during transfer (check file size and line endings)
- If using a config file, validate it is well-formed JSON: `python3 -m json.tool configs/reinit-config.json`

Log files not created

- The script creates the `logs/` directory relative to `os.getcwd()` at startup
- Ensure the current working directory is writable
- If running via `nohup`, the working directory may differ from the script location; use `cd` to set it explicitly before running

--screen fails with "screen is not installed"

```
1 # Ubuntu/Debian
2 sudo apt install screen
3
4 # RHEL/CentOS/Fedora
5 sudo dnf install screen
```

Then re-run with `--screen`.

Can't reattach to the screen session

- List sessions to confirm it is still running: `screen -ls`
- If the session name differs, attach by PID: `screen -r <pid>`
- If the session ended (script finished or crashed), check the summary log under `logs/` for the outcome

Changelog

Version	Date	Description
v2	May 15, 2026	Added <code>--screen</code> flag: auto-launches the script inside a detached GNU screen session to protect against SSH disconnections and terminal timeouts. Implies <code>--bg</code> . Detects existing screen sessions via <code>STY</code> env var to prevent recursion.
v2b	Apr 7, 2026	Parallel peer node operations; end-to-end mode (3); ONTAP upgrade (4a); netboot install (4b); license install (4c); SSH key setup (4d); config backup (4e); BMC auth verify (4f); JSON config file support; background mode; session log with phase/step timing, warnings, and errors inventory.
v2a	Apr 7, 2026	Session logging with timing and summary; warning/error collection in summary;

		<code>_recv_loop + thin wrapper architecture; module-level _peer_reinit_worker.</code>
v1	Apr 7, 2026	Initial release. Modes 1a and 2a.